

HealthApp

A comprehensive implementation review, architecture analysis, and software engineering interview guide.

Reviewed September 23, 2026

This report updates the supplied September 21, 2026 report against the current local application, including uncommitted work. The baseline Git commit is **e39d7c003a9a3324e55d62e15398265b5d088fe0**. That commit alone does not reproduce this review; the evidence manifest records the reviewed working-tree files.

The central design story

HealthApp began as a personal, iPhone-first wellness tracker. Its architecture balances a manageable solo-development workflow with foundations that make additional accounts and devices possible: explicit ownership, relational constraints, transactional saves, bounded external integrations, and feature-focused modules.

Your stated career objective also matters: React and TypeScript provide experience transferable to other application roles. The technical case is separate and concrete: reusable declarative UI, shared domain logic, native platform adapters, and an efficient Windows-to-iPhone development workflow.

The defensible claim: This is a substantial personal application with deliberate reliability and account-isolation mechanisms. It is designed to make future expansion feasible, but its public-launch readiness and maximum capacity have not been established by load or full native acceptance testing.

What this document provides

Current features and implementation paths; alternatives and accepted costs; failure and recovery examples; an updated deployment assessment; interview-ready answer drafts; and a source map that distinguishes code, fresh tests, historical deployment records, and recommendations.

Deliverable scope: An updated report. Application behavior was not changed and no service was deployed during this review.

Reading guide and evidence standard

Use the feature chapters to understand the product, the data-flow chapters to explain reliability, and the interview chapters to practice concise answers.

Pages	Coverage
3-4	Corrections to the original report and current feature inventory
5-8	Architecture, React/Expo choice, backend alternatives, data ownership
9-10	Transactions, retries, conflicts, offline vitals and incremental sync
11-14	Authentication, Face ID, account deletion, onboarding and Profile
15-20	Vitals/HealthKit, training, food, recipes/barcodes and both AI flows
21-23	Calorie/fluid methods, goal persistence and calendar semantics
24-29	Summary, streaks, reminders, photos and export
30-33	Verification, deployment, scaling and prioritized improvements
34-36	Interview narratives, technical follow-ups and decision checklist
37-38	Code evidence map, references and terminology

Four kinds of claim

Implemented: Reachable UI and its corresponding domain, persistence or server path were inspected. This does not mean every possible interaction was tested.

Freshly verified: A check executed during this review. The verification chapter identifies exact commands, results and limitations. Browser tests use synthetic fixtures; isolated SQL uses platform adapters.

Documented deployment: Repository records report a historical hosted deployment or hosted check. This review did not inspect live credentials, query real health records, verify secret values, or redeploy those services.

Rationale or recommendation: Comparative reasoning explains why the current design is defensible, or how it could improve. Unless attributed to an ADR or your stated intent, it is retrospective engineering analysis, not a claim that every alternative was experimentally evaluated when the choice was made.

How to use the interview material

Adapt the first-person drafts to work you personally understand and can demonstrate. Separate your original motivations from lessons learned during implementation. Discuss actual tests and failure cases; do not substitute hypothetical scale, user counts, latency or cost savings for measurements.

What changed from the original report

The September 21 report described commit 4765abd. Several findings remain valid, but major product and lifecycle descriptions are now obsolete.

Original statement / topic	Reviewed current state
Account deletion missing	Now implemented: password-verified preparation, guarded cleanup, retry token, Storage/Auth deletion, device cleanup and recovery UI. Deployment of migration 202609220002 and delete-account v1 is documented.
Four Edge endpoints; 33 migrations	The current repository contains five function entry points and 36 SQL migration files. The added endpoint is delete-account. File count alone does not prove hosted application.
Eleven selectable streak habits	Only calorie target, protein target, fluid target and training consistency are offered. Logging/reminder habits are retired from the UI and saved layout selection.
Calorie streak needs food-day confirmation	The confirmation control is removed. Nonempty saved nutrition totals are evaluated against dated rules/targets; incomplete intake is still possible. Legacy confirmation tables/functions remain.
Optional Recent PR widget	Removed from the current widget registry. Training-analysis code still exists; its existence is not evidence of a visible PR card. Today's Meals is an optional current widget.
Weight/BP logging streaks and import opt-in	No current logging streaks. Manual and imported readings feed combined vital/goal evidence; old import flags no longer justify excluding those readings.
Every reminder is a finite occurrence list	Stable schedules now use native repeats; Workout uses 28 local days; exceptional custom schedules use 14 occurrences. Routine Reminders add five categories.
Fluid saved-drink management visible	The current Water screen uses common/recent drinks and a category popup. Earlier My Drinks creation/management UI was removed; stored records and backend support remain.
Goal helper is a transient calculator	Both helpers now persist last estimates separately from applied goals, retain local input drafts, and support explicit recalculation/application. Loss planning centers on a target date.
Flat Profile / pre-account setup	Grouped Profile routes replace the old screen. Optional setup follows authentication; Welcome is first-use only. Logging screens support backdated entry days.
Add an export schema version	Already present: exportVersion is 1. Export completeness and lower-server-cap pagination issues remain; the recommendation should be to evolve/test that schema, not claim it is absent.

Findings that still stand

No demonstrated public capacity; no complete offline workflow for every tracker; no HealthKit workouts/sleep/steps/HRV import; no cloud reminder service; incomplete export coverage; no receipt/tombstone pruning protocol; and physical-device/real-concurrency gaps. Old ADR paragraphs must be read with their later amendments.

The earlier test counts and CI result belong to the earlier revision. They must not be presented as CI certification of today's uncommitted working tree. Fresh results appear on page 30.

Evidence: Original PDF, pages 1-16; current manifests and route registry; docs/STATUS.md; ADR-005, ADR-007, ADR-008 and ADR-009 amendments; data-export/repository.ts.

Current product and feature boundaries

HealthApp is a wellness recording and planning application. Recorded progress reflects the available logs, not independently measured adherence or clinical outcomes.

Feature	Current user-facing behavior and boundary
Accounts and setup	Email signup/sign-in, verification/reset callbacks, remembered identity, optional Face ID, account-scoped onboarding and settings. No current phone/SMS or MFA enrollment UI.
Summary	Eight default widgets; optional meals, training, quick actions and four goal streaks. Local editable layouts, chart ranges and direct widget configuration.
Manual vitals	Weight and paired BP with optional pulse; dated entries, editable manual history, local durability and conflict recovery. Strongest offline write support.
Apple Health	Read-only, foreground import of weight and BP correlations with nearby pulse association. Requires an iOS development/release build with the native bridge.
Lifting and cardio	Sets/ reps/load, unilateral values, exercise order, notes/location text, manual cardio, local drafts, history edits/deletes and a next-session AI planner.
Food and recipes	Meal drafts, private reusable labels, portion conversion, immutable nutrient snapshots, recipes, barcode lookup and reviewable photo/text estimates.
Fluids	Drink categories and consumed volume; nonalcoholic credit, alcohol recorded without credit, unresolved classification pending. No invented nutrition entry.
Goals	Calorie/protein/fluid/weight/BP targets; transparent helpers; saved estimates; dated goal snapshots and explicit application.
Photos	Private processed JPEGs, up to three new attachments per weight entry, signed display URLs, history-linked gallery and deletion.
Reminders	Opt-in Meals/Fluids/BP/Weight/Workout routines plus custom medication/supplement/other reminders and custom completion history. Device-local.
Profile and data lifecycle	Focused settings, Apple Health and sync status, pending-change recovery, ZIP export, security controls, sign-out and account deletion.
Soon tab	Static, theme-aware Coming Soon design. This is a placeholder, not an implemented undisclosed feature.

What is deliberately outside the current product

No general-purpose Coach chat UI, multiweek training program generator, GPS workout recording, direct OMRON/Strava service integration, Apple Health writeback, server push-reminder system, or tested Android release. Supporting legacy code and an Android config do not establish those capabilities.

One important persistence distinction

Cloud history is shared through Supabase. **Device preferences and unfinished work** include Summary layouts, routine/custom reminders, reminder completions, form drafts and HealthKit anchors. **Active authentication** is process-only. Each has a different lifetime; describing all three as simply 'synced app state' would be inaccurate.

Evidence: apps/mobile/app routes; feature repositories; src/lib/supabase.ts; ADR-002 through ADR-009.

System architecture and trust boundaries

The application is a feature-modular client with a managed backend, not a fleet of independent microservices.

1. Native / browser client

Expo Router screens, React state, shared UI, pure TypeScript models and Zod validation.

2. Device persistence

SQLite / IndexedDB vital cache + outbox; AsyncStorage drafts; protected biometric credentials.

3. Supabase boundary

Auth sessions, RLS reads, transactional PostgreSQL RPCs, private Storage and Edge Functions.

4. External adapters

HealthKit on iPhone; Open Food Facts and AI provider requests through server functions.

Responsibility by layer

Screens manage input, navigation, review and feedback. Pure models own arithmetic, schemas, calendar rules and deterministic evaluation. Repositories translate domain objects to rows/RPC payloads. Platform adapters isolate native storage, notification, camera, sharing and HealthKit differences. The server independently checks authorization and data invariants.

An ordinary confirmed tracker save flows **form -> validation/conversion -> durable intent -> authenticated RPC -> transaction -> receipt -> UI/draft completion**. A manual vital first commits cache and outbox locally, then reaches the cloud later. A photo adds a separate object-storage step. AI output becomes a draft, never an automatically completed health record.

Why this shape fits the original scope

A single person can work on one TypeScript application and one SQL migration history. There is no extra Express, Spring or FastAPI server to operate solely to proxy every read. Edge Functions exist where a secret, external service or privileged workflow justifies server code. SQL is the final boundary for relational consistency.

Costs and alternatives

This reduces service operations but concentrates complexity in repositories and PostgreSQL functions. Supabase Auth, RPC and Storage APIs create platform coupling. A separate application API could centralize policy and versioning more explicitly, but adds another deployment, authorization boundary and network hop. The current design is reasonable while feature boundaries and database contracts remain understandable.

No Redux, distributed queue, Redis cache or realtime subscription layer is required to explain the current save flow. Do not draw them into an interview diagram as if implemented.

Evidence: `src/features/*/{model,repository}.ts`; `src/lib/mutations.ts`; `supabase/functions`; migration 202609180004; `app/_layout.tsx` and platform-specific adapters.

Why React Native, TypeScript and Expo

Career relevance was a legitimate motivation. The engineering justification is reusable application structure with native capability and a practical solo-development loop.

React Native is not a browser wrapped around the app

The client uses React's component/state model with React Native views and native integrations. React Native Web supports a useful browser preview, but HealthKit and device APIs need platform-specific code and testing. Experience with components, hooks, state ownership, TypeScript, API integration and testing transfers to React web work; DOM/CSS and native interaction details do not become identical. [R1]

Alternative	Benefits	Why current choice is defensible / accepted cost
SwiftUI / native iOS	Direct Apple API access, native tooling and platform conventions.	Strong alternative for a permanently iPhone-only app. React Native better serves the stated React-learning goal and shared browser/domain workflow, at the cost of bridge/dependency and native-debugging work.
Flutter	Shared mobile UI/runtime and a cohesive widget system.	Viable engineering choice. It would introduce Dart rather than reinforce the intended React/TypeScript skill set; no measured performance comparison was performed.
PWA / web-only	Simple browser deployment and familiar web development.	Does not provide the same native HealthKit/Keychain experience. The app's iPhone integrations justify a native client.
Bare React Native	Greater direct native project control.	Expo supplies coordinated modules/build tooling, while a custom development build still includes HealthKit. Native-package/config changes still require rebuilding. [R2]

Why TypeScript plus runtime schemas

TypeScript catches incompatible interfaces during development. It cannot validate a stored JSON blob, database response, notification payload or model output at runtime. Zod supplies those checks. Database constraints repeat critical ownership/value rules because a client can be bypassed. The layers protect different boundaries rather than making each other redundant.

State and code organization

React state/context, account-keyed providers, repositories and pure modules fit the current product without a global store framework. Local drafts are persisted where needed. This keeps initial dependencies modest, but introduces custom coordination and potentially large screen modules. If cross-screen cache invalidation becomes the dominant maintenance problem, evaluate a server-state/query library before adding more manual refresh logic.

Interview wording: 'I wanted React/TypeScript experience, but chose React Native because this product also needed native health and device APIs. Expo shortened the build workflow; platform adapters let me share domain logic without pretending native and browser behavior were the same.'

Evidence: Root/mobile package.json; platform-specific *.native/*.web/*.ios files; app.config.ts; eas.json. Alternatives are retrospective analysis. [R1-R2], page 38.

Why Supabase and relational PostgreSQL

The choice favors low operational overhead now and explicit ownership/consistency boundaries that remain useful if more users join.

Documented original constraint

ADR-001 explicitly chose hosted Supabase without requiring Docker/WSL2 on the existing Windows setup. Reviewed SQL migrations, CLI deployment and remote RLS verification replaced a container-dependent local stack. Isolated PGlite checks were later added, but do not emulate the complete hosted service.

Relational fit

A workout has sets; a photo belongs to an account and weight entry; reusable foods and recipe snapshots connect to historical entries; a retry receipt belongs to an operation and user. PostgreSQL supplies foreign keys, transactions, constraints, indexes, JSON fields for flexible snapshots, and SQL reporting. RLS adds database-side account filtering for exposed data paths. This is a better explanation than 'SQL scales better' as a universal claim. [R3]

Alternative	What it offers	Tradeoff for this app
SQLite-only personal app	Lowest backend cost; complete local ownership and offline access.	Cloud recovery, multiple devices, account lifecycle and protected provider calls require additional systems later. Current backend is partly an investment in those needs.
Firebase / document store	Managed identity, client synchronization and document-oriented development.	Entirely viable; parent-child ownership, transactional workout replacement and relational reporting would use different modeling/rule patterns. No benchmark proves it unsuitable.
Custom API + PostgreSQL	Explicit service contracts, portability and centralized business logic.	More server operation and deployment work for one developer. Worth revisiting when integrations/team boundaries need a stable dedicated API.
Self-hosted Supabase/Postgres	Infrastructure control and potential portability.	Backups, patching, monitoring and availability become the project's responsibility sooner.

Scaling potential, honestly stated

Owner-scoped rows and indexes avoid redesigning a single global data store when a second account arrives. Independent accounts have separate logical mutation locks. Stateless Edge request handlers can serve more than one account, subject to hosted capacity and provider limits. None of this establishes a tested user count.

Accepted coupling and future exit

PostgreSQL schema/domain concepts are portable; Auth session checks, Supabase RPC transport, Storage policies and Edge deployment are not frictionless to replace. Repositories and adapters make that migration more localized, not free. Keep provider calls server-side and migration history reviewed so future changes have an explicit contract.

Evidence: ADR-001; SQL migrations; src/lib/supabase.ts; feature repositories; HARDENING.md. [R3-R4], page 38.

Data model, ownership and source of truth

The schema separates facts, editable definitions, derived views and operational metadata instead of treating every value as interchangeable.

Domain / tables	Responsibility and important invariant
Identity and setup	profiles; account_setup: Applied goals/names versus account onboarding/preferences. Setup is versioned and server-persisted.
Vitals	vital_samples: Source-aware readings, occurrence time, database version, deletion marker and change sequence.
Training	workout_sessions; workout_sets; cardio_entries: Parent sessions, ordered exercise/set detail and independent cardio records; child ownership matches parent.
Food	Food profiles, recipes and nutrition entries: Editable private definitions and immutable reviewed/consumed snapshots. History does not depend on today's label values.
Fluids	hydration_entries; saved_drinks: Raw volume, classification and versioned contribution policy; saved definitions do not rewrite old intake.
Photos	progress_photos; private objects: Relational metadata and separately stored image bytes under owner paths.
Streaks	Streak activations/rules/goal snapshots: Effective dates and evidence rules. Legacy food_day_completions remain dormant for current UI semantics.
AI	coach_*; AI usage tables: Private context/messages/actions and atomic quota accounting. Broader legacy storage is not a current chat feature.
Consistency	Receipts, deletion IDs and vital counters: Retry identity, deletion preservation and incremental traversal.
Account security	Device credentials and deletion jobs: Hashed device/recovery secrets and lifecycle state; sensitive administrative access.

Defense at more than one boundary

RLS restricts ordinary account-owned reads. Critical write paths are routed through allowlisted RPCs; grants for unsafe legacy direct writes are removed. SECURITY DEFINER functions need explicit identity/session checks and fixed search paths because they execute with elevated privileges. Composite foreign keys stop a valid child owner from referring to another user's parent.

The ownership rule has intentional exceptions: profiles use id as account identity; shared food catalog/lookup rows are provider reference data; administrative receipt/credential tables have restricted access. 'Every table is publicly readable under user_id filtering' is not an accurate description.

Why snapshots and versions coexist

A snapshot preserves what a meal or goal estimate meant at the time. A version detects a competing edit. A provenance field explains where data originated. An occurrence timestamp answers when the activity happened; creation/update times answer when it was recorded. None substitutes for the others.

Account/date and change-sequence indexes support these queries; execution plans and realistic load still need measurement.

Evidence: 36 files under supabase/migrations, especially 202609180003-005, 202609210001 and 202609220002; feature schemas and mapping functions.

Saving safely: retries and conflicting edits

The central guarantee is an idempotent application mutation with explicit conflicts, not magical exactly-once delivery across every service.

The implemented online save protocol

1. A repository validates intent and creates a canonical fingerprint. The client persists a UUID operation ID and frozen request before sending. A changed form cannot overwrite an earlier operation whose outcome is uncertain.
2. `commit_health_mutation` verifies the authenticated account and active session, locks the operation identity, checks any existing receipt, then serializes writes for that account using the Auth user row. Reusing an ID with a different request is rejected.
3. The transaction validates ownership, expected versions and allowed fields. Workout replacement writes the session and all sets atomically. Goal changes compare only changed fields against their original values, allowing independent field edits to merge.
4. The transaction stores the authoritative response receipt with the data changes. A retry returns that result. For draft-based trackers, the client retains accepted state until durable draft clearing finishes.

Concrete failure example

A user saves a 250 mL fluid entry. The database commits, but the response is lost. The UI cannot infer failure from the timeout. On retry, the same operation ID retrieves the original receipt, so no second 250 mL row is created. A deliberate later identical drink receives a new ID and is a separate event.

Two devices edit workout version 4. The first commits version 5; the second receives a conflict rather than replacing newer sets. Its form remains available for review. A deleted vital cannot be restored by submitting an update against its tombstone; intentional recovery creates a new reading identity.

Why this instead of simpler alternatives

Last-write-wins is easy but silently discards user work. An upsert alone prevents some duplicate IDs but does not freeze intent or validate an edit base. UI button disabling prevents double taps in one process, not lost responses or a restarted app. Transactions and receipts address those distinct failure modes.

Accepted costs

Per-account serialization reduces race complexity but limits simultaneous writes for one account. Receipts and tombstones grow without pruning. The UI must explain conflicts and pending saves. Photo bytes, native notifications and paid AI calls remain outside the PostgreSQL transaction; the receipt guarantee must not be extended to them.

Evidence: `src/lib/mutation-model.ts`, `mutations.ts`; `training/repository.ts`; `goals/repository.ts`; migration 202609180004; `HARDENING.md`. Lock semantics: [R4].

Offline vitals and incremental synchronization

Offline support is intentionally asymmetric: vitals are locally durable writes, while other trackers retain drafts and retry state but need server confirmation.

Local atomicity

On native devices, SQLite stores an account cache payload and an account outbox payload in two tables. The adapter serializes work and uses an exclusive transaction to update both. This is currently account-sized JSON inside SQLite, not a fully normalized row-per-reading cache. Browser storage uses an IndexedDB account transaction. Failed queue persistence must roll back the corresponding cached change.

Local BP pairs are saved together, but each cloud reading has its own operation. An interrupted sync can therefore leave a temporary partial remote pair until retries finish; do not describe the pair as one cross-device atomic write. The pairing/evidence rules avoid treating an incomplete pair as complete BP evidence.

Upload and reconciliation

Queued operations keep their original base version and dependency chain. A successful acknowledgment removes only the acknowledged operation, advances dependent bases, and preserves newer local edits. A downloaded tombstone is not overwritten by an older acknowledgment. Conflicts stay visible for explicit accept-remote or reviewed recover-local actions.

Why the change cursor is not just updated_at

PostgreSQL allocates per-user change_seq values under a counter lock held through commit. A traversal fixes an upper bound and reads ordered changes through that bound. Each remote page and its new cursor commit together locally. Changes beyond the bound arrive next pass. This avoids treating wall-clock timestamps or uncommitted sequence allocation as a reliable visibility order.

History lists use immutable ID keysets and continue until an empty page, even if a service returns fewer rows than requested. This avoids offset shifts after deletion. It is not a multi-table snapshot: a concurrent insertion behind the current ID can appear on the next full refresh.

Tradeoff and next step

Building a full offline engine for every tracker would expand conflict/reconciliation work substantially. The current split prioritizes quick manual vitals and recoverable online saves. Cold-start password/Face ID login needs connectivity because sessions are not persisted; offline use assumes an existing process session.

For long histories, normalize SQLite cache/outbox rows, bound visible queries and profile memory. Any tombstone/receipt retention policy must include a reset/full-reconciliation protocol for long-offline clients; simply deleting old markers could permit data resurrection or duplicate retry.

Evidence: vitals/sqlite-store.ts, store-model.ts, storage.web.ts and sync.ts; src/lib/pagination.ts; read_vital_changes in migration 202609180004.

Authentication, sessions and privacy locking

The app distinguishes establishing identity, protecting an already-open interface, and persisting health records.

Email account flow

Supabase Auth handles email/password identity and verification/recovery. The app validates signup fields, handles allowed deep-link callbacks, supports password reset, and retains remembered-account identity without retaining a reusable password. Signup has an eight-character client minimum; existing sign-in is not incorrectly subjected to signup validation. Hosted password, mail-delivery, session and redirect settings remain configuration dependencies.

Process-only authentication

The Supabase client sets `persistSession` to false and `autoRefreshToken` to true. An active session lives in the process; restarting requires password login or an online biometric exchange. Legacy persisted tokens are removed. This trades cold-start convenience/offline sign-in for a smaller persisted session surface. It is not a claim that health caches or drafts are absent from disk.

Auth providers and asynchronous operations are account-guarded. Private navigation is keyed to the account so state from one user does not simply remain as another user's UI. Confirmed revocation on foreground can sign out the old session; network failure alone does not erase offline work or replace a newer identity.

Privacy lock versus sign-in

Face ID/privacy locking overlays the existing navigator rather than destroying it. Background/resume can protect the view while preserving the current route and unsaved form. Private dialogs participate in the privacy boundary. Unlocking this gate is distinct from issuing a new remote session.

Sign-out semantics and remaining exposure

Normal sign-out uses local session scope; global sign-out revokes biometric credentials and requests Auth global sign-out. Revoking one biometric device stops future exchanges, not every already-issued read capability. Critical writes and incremental vital reads check the actual session row. Ordinary SELECT policies and previously signed photo URLs can retain access until token/URL expiry. [R5]

What not to claim

No general MFA enrollment flow, universal immediate revocation, end-to-end encrypted health history, or application-level encryption of every SQLite/AsyncStorage item is established. The generic `secureStoreAdapter` falls back to AsyncStorage on web or native storage failure; recovery-marker storage therefore needs explicit platform review. The dedicated biometric credential path uses its own protected native handling.

For a wider audience, review backups, device protection, mail security, session policy and redacted observability together; a Face ID button alone is not the complete security model.

Evidence: `src/lib/supabase.ts` and `secure-store.ts`; `auth/auth-provider.tsx`, `auth-callback.ts`, `biometric-lock-navigation.ts` and `security-settings.tsx`; private layouts; `HARDENING.md`.

Face ID device credentials: a real tradeoff

This is a custom server-mediated authentication mechanism, not simply a persisted refresh token behind an unlock prompt.

Enrollment and exchange

The server independently verifies the current password against Supabase Auth and confirms that it belongs to the bearer user. A random 256-bit credential is bound to that user and installation marker. The database retains a SHA-256 hash; the client retains the secret in biometric-protected, device-only, passcode-required SecureStore/Keychain storage. The installation ID is random software identity, not hardware attestation.

On Face ID sign-in, the protected read releases the secret. biometric-auth verifies its hash, device binding, revocation and original 90-day expiry, then atomically consumes/rotates it and returns a new credential plus an Auth session. The client saves the replacement before accepting the session. Rotation does not extend the original expiration.

The client deletes the now-invalid old Keychain item before creating its replacement, avoiding the second prompt associated with updating an existing protected item. A crash in that gap or a lost exchange response requires password sign-in/reenrollment. Accepting the old credential again would undermine replay protection.

Alternatives and why this needs an honest answer

Persist an ordinary refresh token behind biometric access: simpler, closer to standard session lifecycle, easier to maintain. It changes the desired process-only session design and still needs revocation/storage review.

Use password only: smallest custom authentication surface, but worse daily convenience. **Adopt platform/passkey authentication:** potentially preferable as a future standards-based option, depending on hosted support and product requirements; not implemented here.

The existing mechanism supports explicit per-device listing/revocation and avoids storing passwords. Its cost is substantial custom security logic, server coupling and difficult network/Keychain recovery cases. Do not present that complexity as automatically better or necessary for scale.

Evidence and residual work

Tests cover password proof, binding, rotation, replay, expiry, failure and account guards. Repository records report hosted disposable-account verification of deployed biometric-auth v6. Native prompt behavior, biometric enrollment changes and device protection still require the iPhone; a later prepared error-response refinement is not proven deployed by those records.

Interview wording: 'I can explain the credential lifecycle and its failure policy. If I were rebuilding for a broader audience, I would re-evaluate whether a supported standard authentication flow reduces maintenance and security risk without sacrificing the product requirements.'

Evidence: auth/biometric-auth.ts, biometric-credential.ts and security-settings.tsx; functions/biometric-auth/index.ts; _shared/biometric-*; migration 202609180003; HARDENING.md.

Account deletion as a recoverable workflow

Deletion now spans database records, object storage, authentication and device state. A single transaction cannot cover all four.

Preparation establishes authority

The user supplies the current password and deliberate DELETE confirmation. The client creates a recovery token from two UUIDs and persists the pending marker. The Edge endpoint verifies bearer identity and independently verifies the password against that identity. The server hashes the token and calls a service-only preparation RPC.

Preparation serializes with account writes, revokes device credentials, installs the deletion-job marker and removes Auth sessions. Restrictive policies and write guards block further account data access/writes through covered paths. Recovery must then work without the now-revoked ordinary session; possession of the saved capability token identifies the prepared job.

Cleanup proceeds in retryable stages

The service enumerates Storage objects themselves, including owner/path matches, so missing photo metadata does not hide orphan files. It removes bounded batches, then deletes the Auth user, allowing owner relationships to cascade. The finish RPC verifies that Auth, profile, user-owned rows and Storage objects are gone. Only then does it record completion and remove the account identity from the retained receipt.

The client drains ongoing sync, cancels owned notifications, removes local readings, drafts, account keys and biometric credentials, clears disposable private cache, signs out and clears the pending marker. A lost response or partial failure is a reason to resume, not to assume completion.

Cancellation has a precise boundary

An unstarted token can be sealed as cancelled on the server. This prevents a delayed prepare request from starting deletion after the device believes it cancelled. Once preparation has started, the user can leave/sign out and resume later, but cannot reverse completed deletions. Missing endpoint errors are distinguished from an unstarted job so a device is not indefinitely trapped behind a false recovery state.

Why this design

A cascade alone cannot remove Storage bytes or device caches. Deleting Auth first can strand the authorization needed for cleanup. A staged workflow resembles a small saga: each step is retryable, and the capability survives session revocation. Its costs are privileged endpoint logic, retained receipts and careful capability handling.

The endpoint is documented as deployed without gateway JWT verification because recovery can be session-free; that is **not** unauthenticated arbitrary deletion. Initial preparation proves identity/password; continuation proves token possession. No promise covers remote erasure of another offline device, provider records, or backup retention.

Evidence: `auth/account-deletion.tsx`; `functions/delete-account/{index,cleanup}.ts`; migration 202609220002; `scripts/check-account-deletion-hosted.mjs`; `STATUS.md`.

Onboarding, Profile and interaction design

Recent changes replace large mixed-purpose forms with account-aware setup and focused settings destinations.

First use and authenticated setup

The Welcome introduction is a first-use device experience. Account creation/sign-in precedes optional setup. `account_setup` records name, units, current step, completion/dismissal and a version; returning users keep their completed state. The five setup steps are name, goals, fluids, convenience and summary. Skipping is supported without inventing completed health activity.

Onboarding reuses the same calorie/fluid GoalHelper as Profile. It shows applied goals separately from the last calculated estimate. Applying a helper result changes only the intended fields, preserving unfinished protein or target-weight edits. This avoids two calculators with drifting rules and different save semantics.

Keyboard behavior is a layout ownership problem

The updated SetupFrame uses one scroll view with native keyboard insets; actions move with the form rather than competing in a pinned footer. Step transitions reset scroll and dismiss editing. The helper retains stable header/tabs with the same single scrolling owner. Shared inputs receive a scoped iOS Done accessory for numeric keyboards. Short-browser-viewport tests verify bounded fields/actions, but do not simulate a real native keyboard.

Grouped Profile

The landing page exposes Personal Details, My Goals, Preferences, Health Data, Account & Security, separate Sign Out and Manage Account. Nested routes preserve landing scroll; focused sheets handle short edits. Names draw from existing profile/Auth/setup records rather than introducing another independent identity store. Unsaved changes and destructive actions have explicit handling.

Apple Health connection/import and account synchronization are different settings. Sync & Pending Changes surfaces queued vitals and uncertain online saves. Export retains its archive flow with an optional-photo choice. Appearance supports System/Light/Dark; display units preserve canonical values rather than repeatedly rounding conversions.

Tradeoffs

Focused destinations reduce competing form state and keyboard compression, at the cost of extra navigation. Sharing components improves consistency but makes a regression in a common sheet/input affect several features. Browser previews enable fast iteration; VoiceOver, Dynamic Type, native detents, safe areas and keyboard transitions still require actual iPhone verification.

The design aims for explicit save/cancel boundaries and readable errors, not hidden autosave of consequential changes. A user should be able to explore a helper or widget configuration without unintentionally changing a goal or initializing historical tracking.

Evidence: `features/onboarding/*`; `profile/profile-landing.tsx` and focused screens; `goals/goal-helper.tsx`; `ui/keyboard-input-scope.tsx`, `settings-sheet.tsx`; migration 202609210001.

Vitals, charts and Apple Health import

Source identity is preserved in storage; deduplication for display is separate from deleting or rewriting source records.

Manual logging and chart interpretation

Weight and BP inputs validate units/values and accept an entry day. BP stores paired systolic/diastolic readings with optional pulse. Manual records support history edits/deletion; imported readings are presented as source records. Trend windows aggregate the relevant observations. SVG curves use monotone slope limiting so visual smoothing does not overshoot observed values and imply invented extremes.

Combined charts collapse an identical value/kind within five minutes, preferring manual data. Full History retains both origins. This is a conservative duplicate-display heuristic, not proof that two nearby readings represent one event. BP category labels are wellness interpretation, not a diagnostic workflow.

Native import sequence

Connect Apple Health requests read access for body mass, systolic/diastolic BP and heart rate. The reader starts from a one-year lookback and uses anchors for weight and BP correlations. Stable IDs combine account, kind and external UUID; BP members share a correlation identity. Source names and external IDs persist.

The import synchronizes cloud vitals first, reads HealthKit, queues new samples/deletions, syncs again and only then advances saved anchors. Failed final sync retains work for retry. Therefore the implemented import path needs successful database synchronization; it is not an entirely offline HealthKit ingestion engine.

Pulse is selected from up to 12 nearby samples within two minutes, preferring the same named source and then nearest time. It is heuristic, not guaranteed monitor-paired pulse. Weight and BP deletions are anchored; there is no separate general heart-rate import/deletion stream. Pulse lifecycle and association deserve device-level edge-case testing.

Scope and tradeoffs

Read-only import reduces writeback loops and privacy scope. Keeping origins separately makes reconciliation auditable. Device/account-scoped anchors avoid treating a cursor from one Health store as another device's state. Apple deliberately limits what an app can infer about denied read access; 'no samples' is not proof of permission denial. [R6]

Foreground/focus import keeps implementation and battery behavior simpler, but does not promise continuous background freshness. Workouts, sleep, steps and HRV are not imported. The configured purpose string still mentions workout data and should match actual reads before release. No native HealthKit access exists in the browser, though it can display previously cloud-synced Apple-origin records.

Evidence: vitals/manual-vitals-log.tsx, chart.ts, blood-pressure*.ts; domain/vitals.ts; healthkit/reader.ios.ts, sync.ts, identity.ts and permissions.ts; ADR-002; app.json.

Lifting, cardio and deterministic guidance

The training model records actual sessions separately from plans and keeps detailed exercise data under transactional ownership checks.

Lifting workflow and representation

Users choose muscle groups, add exercises, enter ordered sets/reps/working load, reorder exercises, and record notes and a gym/location label. The location is text, not GPS. Unilateral exercises store left/right values while counting one logical set. Explicit zero load is valid for bodyweight work; blank load is not silently converted to zero.

The workout draft is account-local and persists through navigation/restart. Holding a card enables drag/reorder with edge scrolling; accessibility ordering actions offer an alternative. Exercise identity, order and edited set values survive reordering. Save/replace sends one versioned transaction for the session plus child sets; malformed sets roll back the transaction.

Cardio and shared form structure

Cardio records activity/name, duration, optional distance, date and notes. Lifting/Cardio share stable Entry date, Plan workout with AI and type-switch positions; switching preserves their independent drafts. Positive duration is required for cardio to count as completed training. The app does not infer lifting duration from set count.

History-based guidance is a small rule system

Exercise suggestions and remembered performances use the account's own records. `getExerciseGuidance` looks back 90 days with a 100-row cap. `buildExerciseGuidance` chooses the strongest supplied session; it flags an increase only when at least two sets at the working load all reached 12 reps. The current rule suggests +2.5 lb at loads up to 50 lb, otherwise +5 lb. It is not a learned adaptation or universal training prescription.

Full workout/cardio histories use pagination; suggestion queries intentionally remain bounded. Summary training totals count distinct local training days, valid lifting sets and positive-duration cardio. PR-analysis code normalizes case/whitespace and canonical loads, separates side modes and excludes ties/first baselines, but its former optional PR widget is retired.

Alternatives and accepted limits

A simple session note would be cheaper but cannot support set-based history, unilateral detail or reproducible guidance. Automatic HealthKit workout import would provide broad duration/type data but cannot supply this exercise/set/rep/load model reliably. Detailed manual logging costs more effort; reusable drafts and suggestions reduce that burden.

Cloud-confirmed saves need connectivity. Guidance has bounded historical coverage; exercise names are not a complete equipment taxonomy. Generalizing progression should follow real training evaluation rather than adding increasingly confident rules around ambiguous names.

Evidence: `training/repository.ts`, `workout-draft.ts`, `cardio-draft.ts`, `progression.ts`, `exercise-set-fields.tsx`; `summary/training.ts`; `workout.tsx`; migrations 202609150001 and 202609180004.

Food logging and reproducible nutrition

Nutrition history records what the user accepted at logging time, even when the reusable food definition changes later.

Meal construction

A local meal draft contains individual foods and a breakfast/lunch/dinner/snack selection. Inputs can come from manual entry, a private label, recent history, a barcode, an AI estimate or a recipe. Calories and protein are required; carbohydrate, fat, fiber, sugar and sodium can remain unknown. Incomplete optional data is not silently treated as zero in complete totals.

FoodBasis describes a serving and its nutrient values, plus known mass, liquid volume or countable household conversion. Consumed amount is separate. Pure functions compute serving count and nutrients; display unit choices do not replace the canonical basis.

Example of the arithmetic boundary

A label has 120 kcal and 5 g protein per 30 g serving. Logging 45 g means 1.5 servings, 180 kcal and 7.5 g protein. A later label correction to 130 kcal does not change that historical 180-kcal entry. Editing the consumed amount uses the entry's reviewed basis and creates a versioned history edit, rather than mutating the reusable label as a side effect.

Supported units include serving, household count, g/oz/lb and mL/fl oz/cup/tbsp/tsp. Converting mass to liquid volume requires a known basis; the app does not assume every food has water's density. Countable items retain a household quantity/unit so '8 dumplings' stays editable as items rather than becoming a mysterious fraction of a label.

Identity and persistence

Names alone are insufficient to identify a food. Brand, source, barcode/provider identity and a normalized content key help prevent accidental merging. Account-owned corrected labels take precedence for future use. Completed entries retain their nutrient/portion snapshot and provenance; archiving a label removes a picker choice without deleting history.

Saving the meal uses durable intent and server confirmation. The label-saving choice is distinct from logging a food, including mixed choices within one AI-reviewed meal. Retired food-day confirmation functions may still exist in SQL, but the current client does not require or expose that step.

Why snapshots instead of live joins

Live joins reduce duplicated fields but let a future label edit rewrite yesterday's intake. Snapshots duplicate some data while preserving a stable audit trail and predictable totals. This is appropriate for historical facts; reusable definitions remain separately editable. It also means historical correction must be explicit rather than a silent cascade.

Evidence: nutrition/model.ts, repository.ts, draft.ts and history-totals.ts; food-editor.tsx; nutrition.tsx; ADR-004 and ADR-006.

Recipes and barcode lookup

These are reusable-input features, not independent nutrition engines. Both feed the same reviewed portion and snapshot model.

Recipe implementation

A private recipe stores a name/description, declared serving yield and ingredient snapshots assembled in a recipe-only draft. Summed ingredient nutrients divided by yield define one recipe serving. Decimal servings or a fraction of the whole recipe reuse the existing portion calculator. A four-serving recipe with 1,200 kcal totals 300 kcal per serving; half the recipe is two servings, 600 kcal.

Optional nutrient completeness propagates from every ingredient. Recipes cannot contain recipes, avoiding circular/nested composition and opaque recalculation. Archival removes future selection but preserves historical references. Reproducibility wins over automatically updating all recipes after a label correction; a revised preparation is an explicit new/replaced definition rather than an invisible cascade.

Barcode request path

Expo Camera or manual entry supplies digits/type. The client checks the user's private foods first. An authenticated resolve-food-barcode function normalizes the identifier and consults shared product/lookup caches before calling Open Food Facts. Provider output is normalized and validated into the same serving/nutrient representation.

The resolver has an eight-second provider timeout, approximately 30-day positive cache expiry, 24-hour not-found caching and a usable stale-product fallback. Shared catalog data is external reference information, not a public copy of a user's meal history. The user confirms uncertain package/serving information and can create a private correction.

Why a server cache and private override

Direct provider calls from every screen duplicate normalization and consume upstream requests. A server cache centralizes behavior and improves repeat lookup latency; a negative cache avoids repeatedly asking about missing products. Private overrides preserve personal corrections without asserting that one user's label should replace the shared source.

Alternatives and failure policy

Bundling a global product database creates large update/storage work. A paid provider may improve coverage but adds contract/cost dependencies; USDA search is later work, not current fallback. If a barcode is invalid, missing or ambiguous, manual entry remains available instead of inventing a product match.

Caching trades freshness for fewer requests. A recognized barcode is not proof of label accuracy, region/packaging match or correct portion size. Future wider use needs upstream attribution/license review, burst controls and measured hit/miss/timeout rates; no current benchmark establishes cache savings.

Evidence: nutrition/recipe.ts, recipe-manager.tsx and repository.ts; functions/resolve-food-barcode/index.ts; _shared/barcode-normalization.ts and food-normalization.ts; ADR-006.

AI meal estimates: bounded, reviewed input

The model helps decompose a meal. The application remains responsible for validation, portion arithmetic and whether any record is saved.

Request and response flow

The user supplies a photo and/or description with explicit processing consent. The client prepares a bounded JPEG input. The Edge handler authenticates, streams a size-limited request, validates schema/consent, consumes an atomic per-user quota and calls the provider. It does not query personal health history for this feature.

Code limits include 20 estimates per user per day, images up to 2 MiB, at most 15 components, a 60-second provider timeout and 3,500 output tokens. The fallback model string is gpt-5.4-mini, overridable by a server secret. This identifies the code default, not the deployed secret value, model access, actual price or measured quality.

Structured output classifies the input as meal, non-food or unclear, then returns components, serving bases, consumed amounts, nutrients, confidence and assumptions. Server/client validation rejects malformed output. Name normalization and draft checks reduce duplicate components when photo and description describe the same meal.

Human review is part of the architecture

The user can edit/remove components and choose reusable-label storage per item. Cooking-fat assumptions remain visible; explicit dry/oil-free preparation changes the instruction. Reviewed items enter the normal meal draft. Completed history still requires the normal Save path. The model does not receive a mutation tool that can bypass it.

Tradeoffs and alternatives

Manual entry is more controllable but slower. An unconstrained chat response is easy to display but hard to validate and edit reliably. Structured output plus deterministic arithmetic supports predictable UI while still permitting portion/ingredient errors. A custom recognition model would require a substantial dataset and evaluation effort; a hosted model reduces initial work but adds latency, privacy and variable cost.

The request uses store:false and avoids logging raw images/meal bodies. That is not a blanket guarantee about provider retention. The meal handler consumes quota before the call and has no matching failure-refund path; failed attempts may still use the app's daily allowance. There are no paid automatic retries.

What evidence is still needed

Evaluate representative cuisines, hidden oils, mixed dishes, unclear images and adversarial/non-food input. Measure nutrient/portion error and user correction rate, not just schema validity. No clinical accuracy or measured superiority over another model is established by existing tests.

Evidence: nutrition/ai-meal-client.ts, ai-meal-image.ts, ai-meal.ts and ai-meal-editor.tsx; functions/estimate-meal/{index,handler}.ts; _shared/meal-estimate.ts; ADR-004.

AI workout planning and retained Coach code

The current product generates one editable session from a questionnaire; broader Coach backend capability must not be confused with a visible chat product.

Current interaction

Plan workout with AI opens a three-step preference flow for a lifting, cardio or mixed session. It gathers goals/styles, experience, available time, equipment, focus, limitations and readiness with training-history consent. Multi-select arrays are validated; no-equipment is exclusive; legacy singular preferences are normalized at the boundary.

Generate authorizes reversible population of local Lifting/Cardio drafts. Existing affected drafts require an explicit replacement choice. A rest recommendation is shown without creating activity. Successful output includes exercise/set detail, effort/rest/technique and relevant notes; normal tracker saves are still required to claim a completed workout. Entry-date drafts remain protected.

Server context and validation

For these requests, coach-chat uses training-only context, including bounded recent sessions/sets/cardio and deterministic 7/28-day primary-muscle-group summaries. It disables nutrition, hydration, vitals and photo context despite broader legacy profile permissions. Training-only tools and next_workout action checks restrict what can be proposed.

Shared guards verify requested sections, duration budget, exercise shape and effort/technique constraints. A plausible-looking response can still be rejected. This is defense beyond prompting, but does not prove biomechanical safety or optimal programming.

Cost and legacy boundaries

New planning uses the deep tier: 3 requests per user per UTC day and a 4,000-token output cap. Retained standard routing allows 30 requests/day and 1,400 output tokens. Code defaults are gpt-5.6-terra for deep and gpt-5.6-luna for standard, with server overrides. Tool rounds, history and per-request context are bounded; provider calls have timeouts. Coach has failure-refund logic for its app quota, unlike the meal estimator; this does not refund provider billing.

Private profiles, threads/messages/actions, broader tools and explicit research support remain for legacy/backend compatibility. There is no current general chat composer/history selector in the planner UI. Documented deployment says coach-chat v15 was activated; this review did not inspect live model configuration.

Alternatives and next evaluation

A deterministic template library is cheaper and more predictable, but less flexible across preferences. An autonomous 'coach' that writes workouts would remove meaningful user review. The chosen middle ground is flexible generation with a bounded action schema and reversible drafts.

Before broader rollout, evaluate plan validity, time realism, exercise suitability, refusal quality, history overreliance and replacement behavior. Keep plans distinguishable from completed exercise; do not use an AI generation count as adherence evidence.

Evidence: app/(app)/coach.tsx; coach/workout-questionnaire.tsx and draft-actions.ts; functions/coach-chat; _shared/workout-planning.ts and coach-action-validation.ts; ADR-005 amendments.

Calorie helper: exact arithmetic and limits

This is a transparent local planning calculation with a saved estimate, not a personalized metabolic measurement or guaranteed weight forecast.

Current equation

The app uses simplified Mifflin-St Jeor: $10 \times \text{kg} + 6.25 \times \text{cm} - 5 \times \text{age} + s$, where s is +5 for the male equation and -161 for the female equation. The UI calls this BMR; the underlying publication predicts resting energy expenditure. The estimate is multiplied by the selected activity factor: **1.2, 1.375, 1.465, 1.55, 1.725 or 1.9**. [R7]

The code validates age 19-120, height 100-250 cm and weight 30-500 kg. These are software input bounds, not proof that the original equation was validated across all those ages/body sizes. Exact conversions use 0.45359237 kg/lb and 2.54 cm/in; intermediate values remain unrounded and displayed/saved calories are whole kcal. Workout calories are not added again on top of the activity estimate.

Worked example, not a recommendation

For synthetic inputs age 30, 80 kg, 180 cm and the male equation: $800 + 1,125 - 150 + 5 = \mathbf{1,780 \text{ kcal}}$. At factor 1.55, maintenance estimates **2,759 kcal/day**. Planning 10 lb over 10 weeks means 1 lb/week and an arithmetic 500-kcal daily deficit, producing **2,259 kcal/day**. Real change will not necessarily match that line.

Target-date loss planning

The primary form selects a local target date. Calendar-day duration is 7-3,640 days; exact required loss must be 0.1-2 lb/week. The calculation subtracts rate $\times$ 500 kcal/day. Invalid dates, rates or generated targets produce errors rather than substituting a standard rate. BMI checks and unsupported-scenario messaging remain. Compare Other Options offers standard 250/500/750/1,000-kcal deficits. Maintain and gain use maintenance or the app's +5%/+10% convention.

Generated targets below 1,000 kcal/day are unavailable. The NIDDK planner also warns against below-floor intake, but that does not establish every value above 1,000 as adequate or appropriate. Its dynamic planning model is not implemented here. [R8]

Why this method and what to say

The documented change from DRI EER followed the user's preference for a familiar calculator-style method. Its benefits are understandable inputs, reproducible tests and transparent assumptions. Alternatives include another population equation, a dynamic model, measured expenditure or manual targets. None removes individual uncertainty. Activity multipliers, fixed deficits and gain percentages must not be described as the Mifflin paper validating the entire product.

Evidence: goals/calculator.ts; helper-model.ts; calorie-goal-form.tsx; GOAL_HELPER_EVIDENCE.md. Scientific references [R7-R8], page 38.

Fluid goals, drink policy and protein

The product intentionally measures beverage volume rather than presenting an unsupported physiological hydration score.

Suggested fluid target

Suggested mode starts at **3,000 mL beverages for the male category or 2,200 mL for the female category**, distinguished from the 3,700/2,700-mL total-water references that also include food. These are population references, not personal minimums. [R9]

The app adds 0, 250, 500 or 750 mL for Mostly sitting, Lightly active, Moderately active or Very active. Those increments are transparent product planning allowances, not measured sweat losses or clinically validated individual requirements. There is no current weight multiplier, climate questionnaire or workout-minute addition. Custom mode bypasses those inputs entirely.

Canonical mL is preserved through unit changes; US fluid-ounce conversion is 29.5735295625 mL. The estimate becomes the saved goal only after explicit application. Gender labels map to the equation/reference categories with explanatory text; manual goals remain available rather than pretending the categories capture every person's physiology.

Drink logging and versioned credit

Input policy	Raw history	Goal contribution
Classified nonalcoholic drink	Preserve consumed liquid volume and category	Full logged beverage volume
Alcoholic drink	Preserve raw volume and alcohol classification	Zero, never negative credit
Unknown alcohol status	Preserve entry for later classification	Pending / null contribution
Legacy volume policy	Preserve earlier meaning	Original volume credit until explicit reclassification

Client/server share fluidContribution/fluidTotals; SQL generates counted_ml and validates category/status. Logging coffee or a shake does not automatically create calories/protein in Food. Quick Fluid Amount fills only the amount; the user still selects the drink. The category popup and recent choices replace the former My Drinks management flow without deleting stored definitions.

Protein and other goals

Automatic protein uses the app's rounded **0.7 g/lb** rule from an eligible saved weight, unless manually overridden. Imported and manual readings participate; target weight is not automatically a newly measured weight. BP's stored default 120/80 and protein's multiplier are product settings, not individualized clinical prescriptions. The app supports manual calorie, fluid, protein, weight and BP goals.

Alternative considered

Assigning coffee a '60% hydration score' or alcohol a negative percentage would suggest precision this model does not establish. Recording actual volume with an explicit policy is more auditable, though it deliberately does not measure fluid balance. A future sweat-loss/medical workflow would require separate validation and scope.

Evidence: goals/calculator.ts and repository.ts; hydration/model.ts and categories.ts; _shared/hydration.ts; migration 202609170002; ADR-007; [R9].

Goal state, snapshots and entry dates

A draft, an estimate, an applied goal and a historical denominator are different values with different lifetimes.

Four layers of goal state

Input draft: Account-local values preserve unfinished helper edits. **Last estimate:** helperVersion 2 / latestCalculation persists a validated snapshot in the existing profile JSON, including inputs, canonical values, target, assumptions and relevant dates. **Applied goal:** Numeric profile targets and provenance change only on Use This Goal or an explicit manual edit. **Dated target:** Historical snapshots determine the denominator for streak evaluation.

Reopening shows the saved result. Recalculate is explicit and uses the current date; invalid new inputs do not erase the previous estimate. Manual goal edits preserve the previous calculation while recording custom applied provenance. Importing the latest weight into a calculation is an explicit action. Retries reuse the original operation and timestamps.

Prospective goal changes

The transactional goal path preserves today's snapshot and writes the next effective target, normally tomorrow. Training rule changes start at a Monday boundary. Independent goal fields can merge; simultaneous edits to the same original field conflict. This prevents a changed present goal from silently rewriting every earlier success/failure.

Automatic protein snapshots resolve from the most recent eligible weight for that day, including source identity and numeric result. Already-resolved dates do not adopt a newer weight. Before reliable dated targets exist, the app cannot invent historical qualification from today's settings.

Backdating is about occurrence, not a fake creation time

Weight/BP, food, fluids, lifting, cardio and photo attachments accept a local Entry date, defaulting to today. Future/invalid dates are rejected. entryTimestamp replaces the local year/month/day on the current time, then serializes to an instant; it does not mean UTC midnight and does not provide a full user-chosen time-of-day editor.

Explicit dates survive relevant drafts and AI appends. The selected day participates in mutation identity; the resolved timestamp is frozen for retry. Date grouping uses local calendar arithmetic. Traveling may regroup timestamps into a different local day; a selected date is not a universal immutable home-time-zone policy.

Why explicit layers beat automatic recalculation

An always-changing goal seems convenient but makes historical progress unstable and can overwrite a user's deliberate target. Separate state costs additional metadata/UI, but supports reproducibility, user control and clearer conflict handling. A future timezone policy should be chosen intentionally rather than inferred from UTC timestamps alone.

Evidence: goals/helper-model.ts, helper-draft.ts, use-helper-state.ts and repository.ts; src/lib/entry-date.ts; entry-date fields; streak SQL functions; ADR-008 amendments.

Summary widgets and the Coming Soon tab

The dashboard separates layout configuration from health data, allowing personalization without changing the underlying records.

Registry and persistence

The widget registry declares type, supported size, configuration schema and data needs. Defaults are Weight, Blood Pressure, Calories, Protein, Fluids, Calorie Calendar, Weight Trend and BP Trend. Optional additions are Today's Meals, Training, Quick Actions and Streaks. The old PR widget is retired; reading an old layout removes only retired content and preserves remaining identities/order/settings.

Layouts are versioned, validated, account-keyed AsyncStorage JSON and stay device-local. A serialized store coordinates reads/writes. An intentionally empty layout remains empty. Corrupt/unknown data produces a visible recoverable default preview rather than immediately overwriting stored configuration.

Responsive and editable presentation

Adjacent Small cards pack in order; Large starts a row. The stored legacy size value wide is labeled Large. Small displays full width below 370 points or above fontScale 1.25 without changing the saved preference. The editor/gallery use the same card frames and renderers as the live Summary.

Holding a widget begins drag with a floating card, insertion placeholder and edge scrolling. Stable starting measurements and committing order on release preserve pointer behavior; screen-reader ordering actions remain. Done/Cancel is the persistence boundary. Restore defaults is confirmed. Chart and normal navigation gestures are disabled in editing previews.

Direct configuration and the jitter fix

Live Streaks and Quick Actions have a top-right pencil with a 44-point target. Each opens a cloned layout draft with fixed Save/Cancel. A full Preview widget view is separate from the choice list; changing selected habits no longer changes the height above those choices and moves the sheet. Saving new streak selections initializes only absent habits; previewing does not.

Today's Meals uses saved local-day nutrition entries independently of whether reusable labels were retained. Data categories can fail independently; an unavailable source should not be rendered as zero activity. Focus/foreground refresh provides eventual UI freshness rather than realtime push updates.

The Soon tab is a static SVG sealed-card illustration with Under Wraps / Coming Soon / In the Works text. It adapts to theme and shorter screens, hides decorative art from accessibility, and has no hidden network or feature-unlock behavior.

Tradeoff

Local layouts avoid cloud schema/conflict work for a personal preference, but do not follow the user to a second device. Shared renderers improve consistency while increasing the need to verify each preview/edit interaction context.

Evidence: [summary/layout.ts](#), [layout-store.ts](#), [editor-grid.tsx](#), [dashboard.tsx](#) and [data.ts](#); [extra-widgets.tsx](#); [app/\(app\)/placeholder.tsx](#); ADR-008.

Goal streaks: evidence, not a stored counter

Only four goal habits are offered now. Their results are recomputed from dated rules, dated targets and available source records.

Current qualification rules

Calories: A nonempty saved food day must satisfy the inclusive at-or-under / at-or-over target rule. Older range/tolerance revisions remain readable. No manual food-day completion is required. An under-target check is about recorded totals, not proof that the user logged all intake.

Protein and fluids: Recorded protein or counted beverage volume must reach a positive effective target. Alcohol/unknown classification follows the shared fluid policy. **Training:** Distinct local days with valid saved lifting sets or positive-duration cardio must meet the configured 1-7-day weekly target. Two workouts on one day count as one training day.

Evaluation model

The pure engine accepts sources, coverage, dated rule revisions, goal snapshots and an injected clock. It returns periods with met/open/not_met/unknown/not_scheduled states, numeric evidence and explanations. Missing query coverage or missing historical goals produces unknown rather than silently substituting zero or today's goal.

An unfinished period retains the previous run; qualifying current evidence can provisionally extend it. A closed failure breaks a run. Paused/off periods are neutral. Best excludes the unfinished period, and unknown gaps prevent a verified consecutive claim. Edits/deletions cause recomputation instead of leaving an obsolete cached badge.

Activation and display

Saving a selected streak initializes only absent habits, using available evidence dates without overwriting existing rules. Preview/Cancel does not initialize. Goals cannot be fabricated before reliable snapshots; different evidence histories can still imply different tracking start dates in details.

All cards show the current Monday-Sunday week with initials, checks and day numbers. Training checks show actual training days, not seven copies of the weekly result. Future dates are neutral; unknown and pre-activation/off days remain distinct. The old daily Met field and card-level Since label are removed.

Why derive instead of increment

A stored streak++ shortcut is fast but becomes wrong after backdated logs, goal changes, deletions, missing coverage and travel. Pure recomputation is easier to reason about and test. Its cost is history loading and CPU/memory growth. A future materialized summary needs invalidation rules for every source/rule/goal change, not merely a nightly increment.

Legacy logging/reminder rule rows remain for compatibility, while current loading/layout migration filters them. This avoids destructive deletion of old server state at the cost of extra historical concepts in code and documentation.

Evidence: streaks/engine.ts, initialize.ts and repository.ts; summary/streak-week.ts and layout.ts; migrations 202609180001-002 and 202609220001; ADR-008 amendments.

Routine Reminders and native scheduling

The app chooses predictable, editable local schedules over a server delivery system or health-dependent automation it cannot reliably execute while suspended.

Configuration and local ownership

Meals, Fluids, BP, Weight and Workout routines are opt-in, with master/category/slot controls, times and selected weekdays. Workout days are initially unselected. Quiet hours default to 22:00-07:00 and apply to routines; conflicting times require correction. Within-category duplicates block activation, while simultaneous cross-category times produce a notice.

Validated routine preferences are account-keyed in AsyncStorage. Native request IDs/status are stored separately. Existing custom medication/supplement/other definitions and completion history remain in their own repository. Permission denial retains choices and offers Settings rather than repeatedly prompting.

Schedule type	Implemented delivery model
Stable daily / multiple-daily	Native repeating calendar time; does not depend on JavaScript being open to renew every occurrence
Selected weekdays	Repeating calendar request per selected weekday/time
Workout routine	Dated requests over 28 local calendar days; foreground/login/edit replenishes the horizon
One-time custom	A single dated occurrence
Future-start or early-completed custom exception	Next 14 valid dated occurrences, then reconciliation can extend or return to repeats

Reconciliation, not cancel-all

One serialized service calculates desired requests. Deterministic account/source/group/slot IDs and fingerprints let it compare desired state with the OS pending list. It adds/replaces, verifies with the native API, then removes obsolete owned requests. Existing custom groups migrate first so frequent routines do not silently evict medication schedules.

The app budget is 60 pending requests, with four transition slots for verified replacement. This is a conservative implementation policy, not a claim of a permanent documented OS-wide limit. It counts other-account requests and fails visibly on capacity rather than silently truncating the intended schedule. [R10]

Tradeoffs

Server push could synchronize multiple devices and support centrally evaluated conditions, but adds tokens, infrastructure, connectivity and delivery-policy work. Native repeats minimize that burden for personal use. They can still fire after activity was logged: automatic suppression from current food/fluid/HealthKit/training data is deferred.

Rest Today cancels today's remaining workout prompts and stores a local preference; it creates no completed workout or streak credit. DST/travel and repeating triggers while suspended still need native testing. A 28-day workout horizon is not indefinite delivery if the app is never reopened.

Evidence: reminders/routine-model.ts, routine-storage.ts, schedule-model.ts, service.ts and device.native.ts; ADR-009; Expo notifications [R10].

Reminder taps, drafts and lifecycle races

A notification is a request to navigate, not authorization to open private data or evidence that an activity occurred.

Intent validation

Startup notification responses and the live response listener feed one persisted intent. Payloads use known account/source/category/slot or custom reminder identifiers. Arbitrary URLs, credentials and health totals are not navigation instructions. Legacy IDs resolve only against a unique owned definition.

Response identity, delivery timestamp and action deduplicate startup/listener events without suppressing tomorrow's tap on the same repeating request. An intent expires 30 minutes after capture. Failed login can retry with the intent intact; cancellation, wrong account, expiry, deletion or successful consumption clears it.

Deferred navigation gates

Routing waits for foreground activity, initialized navigation, account setup, authentication and privacy unlock. A final account/unlock check after asynchronous consumption protects against a late session switch or lock. Destinations are a fixed internal set: Food, Fluids, BP, Weight, Workout and Reminders.

Tapping an old prompt does not backdate a log to its old occurrence. A meal hint fills only an empty draft; existing meal selection, entry date and foods survive. A custom reminder tap opens the screen without marking it taken. This keeps user intent separate from data mutation.

Logout and deletion behavior

Explicit sign-out cancels outgoing routine requests and pending navigation while retaining preferences. Activating an account restores its enabled choices and cancels other-account routines. Automatic session expiry preserves intent/settings to allow login recovery. Custom reminder logout behavior remains unchanged: existing custom schedules can continue. That distinction should be disclosed rather than promising all prompts disappear on logout.

Account deletion cancels owned routine/custom/legacy pending requests, dismisses owned delivered notifications and clears the pending intent before local account cleanup. There is no authentication secret in a notification payload.

Why this complexity is justified

Directly navigating inside a response callback is simple until the app is cold-starting, locked, signed into another account or still loading setup. A small validated intent state machine handles those independent lifecycle boundaries. Tests simulate those races deterministically; real delivery, cold starts, OS permissions and Face ID remain device tests.

Structured duplicate detection only offers replacement for matching BP/weight kind/time/weekdays. Medication names are not heuristics for deleting another reminder. The old reminder is disabled only after the replacement schedule is confirmed, preserving the user's existing behavior on a failed activation.

Evidence: reminders/navigation-model.ts, notification-observer.tsx, intent-storage.ts, lifecycle.ts, meal-intent.ts and service.ts; ADR-009.

Private progress photos and object storage

Image processing and access controls bound cost and exposure, but object bytes and metadata do not become one atomic database transaction.

Input and storage model

Camera/library images are re-encoded to JPEG, stripping retained EXIF metadata in the processed output. The first maximum dimension is 1,440 px, with progressive compression/resizing toward 900,000 bytes and a hard 2 MiB ceiling. The stored image is a processed derivative, not the original camera asset.

New uploads require an active owned weight reading and are limited to three per weight entry. SQL serializes the count check for a user/entry to avoid simultaneous fourth attachments. Existing legacy/unlinked or over-limit groups are retained rather than silently deleted. Photo history is tied to `weight_sample_id`, not inferred solely from the date.

Metadata lives in `progress_photos`. Bytes use a non-public bucket and a path beginning with the account ID, followed by date/UUID. Storage policies and table ownership checks protect separate boundaries. Display uses one-hour signed URLs; no public permanent image URL is persisted. A leaked signed URL is still a temporary capability until expiry. [R11]

Retry and deletion path

The upload operation persists a stable path and frozen payload. Retrying can tolerate an already-existing object response rather than creating a fresh object each time. Metadata creation uses the common mutation receipt path. Deletion commits metadata change before attempting object removal; failures remain recoverable through the saved operation path.

A definitively rejected metadata creation can leave an orphan object. There is no automatic background orphan sweep. Whole-account deletion enumerates actual objects and can clean these up, but that is different from ongoing storage reconciliation for active accounts.

Alternatives and why this is reasonable

Storing full image bytes in relational rows complicates database size, backup and read traffic. Public URLs simplify rendering but do not match private progress photos. Full-resolution originals improve fidelity but increase storage/egress and retain more metadata. Private processed objects strike a useful personal-app balance.

Remaining validation

Test camera orientation, permission denial, large images, failed uploads, 409 retries, metadata rejection, expired URLs, low storage and gallery memory on iPhone. Bounded metadata paging and a small rendered image window help, but do not establish performance for an unlimited archive. Future scale needs an orphan-reconciliation policy, photo quotas/budgets and an explicit storage retention plan.

No end-to-end encryption claim follows from a private bucket. Supabase/server access and operational policies remain part of the trust model.

Evidence: `progress-photos/image.ts`, `model.ts`, `repository.ts` and `progress-photo-gallery.tsx`; `src/lib/mutations.ts`; migrations 202609040002, 202609160001 and 202609180005; ADR-003.

Export: useful portability with known gaps

The current archive is valuable, but it should not be described as a complete, consistent or restore-capable backup.

What is implemented

Profile collects account/profile information, vitals, workouts/sets, cardio, nutrition, fluids, saved foods/drinks, recipes, photo metadata and Coach records. Local custom reminders/completions, unfinished food/workout/cardio drafts, HealthKit state and cached vitals are included. `exportVersion` is already **1**.

The archive contains structured JSON, per-category CSV and a manifest, with optional stored JPEGs. `fflate` builds the ZIP. Native code writes chunks to a cache file, shares through Expo Sharing and removes the temporary archive. Browser download uses its platform adapter. Optional photos are the processed stored assets, not original library files.

Confirmed completeness issues

The table list omits `streak_activation`, `streak_rules`, `streak_goal_snapshots` and legacy `food_day_completions`. `account_setup` and newer device configuration are not fully represented; Summary layouts, routine preferences and helper input drafts are not collected by this repository. Applied goals/latest estimates in profiles are different from omitted local helper drafts.

`loadRows` requests ranges of 1,000 and stops when the returned page is shorter than 1,000. If the service caps responses at 500, it returns after the first 500 even when more records exist. This path does not use the `core-history collectPages` loop. The photo count/byte summary uses the same short-page assumption.

Offset paging and concurrent `Promise.all` table reads also do not provide a common point-in-time snapshot. An edit/delete during collection can produce a mixed view. The archive currently has no import/restore workflow, and local device credentials should not be casually added to a portable export.

Recommended repair, not implemented in this review

Define an explicit coverage manifest with included, omitted and device-only categories. Upgrade pagination to a stable cursor that continues until empty, with tests under a lower server cap and concurrent changes. Evolve `exportVersion` when adding formats/categories. For stronger consistency, use a server-defined snapshot/export job or clearly label the export as a best-effort current view.

For large histories, stream/batch collection and photo retrieval, enforce archive-size expectations and test cancellation/low-storage cleanup. A chunked file writer does not mean every upstream dataset is streamed or memory-bounded.

Interview lesson

The presence of an export button is not proof of completeness. The more defensible answer identifies its contract, demonstrates the pagination failure condition, and proposes a targeted fix rather than claiming the entire persistence system is broken.

Evidence: `data-export/repository.ts`, `model.ts`, `archive.native.ts` and `archive.web.ts`; `profile/health-data-screens.tsx`. No export repair was made in this report task.

Verification performed for this review

Tests support particular claims. They do not substitute for untested platform behavior, production configuration or measured scale.

Fresh check on September 23, 2026	Result and scope
npm run check	Passed ESLint, strict TypeScript, 231 app tests and 66 Edge helper/handler tests . The npm lifecycle runs lint before typecheck and tests afterward.
node scripts/check-onboarding-sql.mjs	All 36 migration files executed in isolated PostgreSQL/PGLite. Passed hardening, onboarding, original streak SQL, Summary initialization, permissions, deletion guard and cleanup-verification assertions.
Selected Playwright suites	55 tests passed in one run (reported 2.7 minutes): review, hardening, onboarding, Profile settings, Routine Reminders, deletion recovery, goal helper and Summary refinements.
Report QA	Source reconciliation plus rendered-PDF inspection; document generation checks page bounds and extracted text. This is artifact QA, not an additional app acceptance test.

What the checks actually exercise

Pure tests cover conversion/calculation, portion snapshots, date rules, streak evidence, draft state and reminder planning. Mutation/storage tests exercise durable operation IDs, failed persistence, acknowledgment races, tombstones and conflict resolution. SQL tests execute real migration/function logic under isolated database roles, with minimal Auth/Storage schema adapters.

Browser tests interact through visible controls with synthetic Supabase/provider responses. They cover ordinary saves, error/retry/recovery flows, bounded sheets, account setup, goal application, widget migration/editing, reminder navigation and deletion recovery. They do not certify live RLS or a real provider response.

Checks deliberately not represented as passed

No fresh hosted data or secret inspection; no separate-connection database concurrency run; no load benchmark; no physical iPhone keyboard, Face ID, HealthKit, camera, notifications, accessibility or Storage integration run; no App Store Connect inspection or release build submission. PGLite serializes calls; a full Deno entry-point typecheck was not rerun. A mocked native adapter and a successful Hermes export are not real device acceptance.

Earlier repository records report iOS/Hermes exports, targeted hosted SQL/Auth/deletion checks and light/dark visual review. Those remain historical evidence, separately attributed. The existing GitHub workflow runs npm ci and npm run check; it does not currently gate SQL, browser or native suites. No fresh CI result for the uncommitted tree is claimed.

Reproduction evidence

Root logs: dist/report-review-check.log and dist/report-review-sql.log. Browser log: apps/mobile/dist/report-review-browser.log. The source manifest records commit, dirty-file status and file hashes without secrets or health records. Test counts are not a coverage percentage or defect-free guarantee.

Evidence: package.json scripts; .github/workflows/ci.yml; scripts/check-onboarding-sql.mjs; test logs listed above; apps/mobile/e2e fixtures and specs.

Deployment state and a credible release plan

Code present, backend deployed, native binary distributed and public release approved are separate milestones.

Current evidence

Component	What the repository establishes
Native client	Expo SDK ~57.0.18, RN 0.86.3, React 19.2.3 and TypeScript ~6.0.3 declarations. EAS development/internal preview/production profiles exist; native OTA updates are explicitly disabled.
Hosted consistency/security	HARDENING records migrations 202609180003-005 and biometric-auth v6 deployed with targeted hosted checks. A later local error-response refinement is not established as deployed.
Summary and deletion	STATUS records migration 202609220001 plus exact initialization RPC verification; migration 202609220002 and delete-account v1 deployment plus disposable cleanup/recovery/isolation checks.
AI planner	ADR-005 amendments document coach-chat v15. Older setup documents still describe earlier chat UI/version/costs and must not override current code.
Other provider functions	Meal/barcode entry points and setup/deployment records exist; exact current live versions, secret values, quotas and funding were not rechecked.

Development versus release

An installed EAS development client loads compatible JavaScript from Metro. Native package/configuration changes require a new binary. A standalone production build is the public deliverable. Development and release schemes/bundle IDs are distinct; redirect allowlists must match. EAS Submit uploads to App Store Connect but does not by itself publish the app. [R2, R12]

Staged public-release work

First establish separate intended development/staging/production configuration and inspect Auth policy, SMTP, redirects, bucket policies and provider budgets. Exercise true multi-session/multi-user tests and backup/restore in staging. Build and test the actual release binary with native permissions, keyboard, accessibility and adverse network/storage cases.

Then complete privacy policy, support details, App Store privacy disclosures, accurate HealthKit purpose strings and AI-sharing consent. Account deletion is now implemented; test it in the release binary rather than retaining the old report's recommendation to build it from scratch. Apple's account-creation apps must offer deletion initiation. [R13-R14]

Use a small TestFlight cohort before expansion. app.json advertises tablet support; validate iPad or intentionally change that scope. Coordinate schema and client releases: older direct-write clients were intentionally blocked by hardening. A minimum-supported-client strategy and compatible migration/forward-fix plan are still needed. No deployment or release-setting change was performed for this report.

Evidence: app.config.ts, app.json, eas.json; STATUS.md; HARDENING.md; ADR-005 and ADR-008; SUMMARY_REFINEMENTS.md; [R2, R12-R14].

Future scale without premature infrastructure

Account separation and transaction design create room to grow. Capacity is a workload measurement, not a property implied by choosing React or Supabase.

The personal-use balance

Keep the managed architecture initially. Additional services are justified by observed bottlenecks or ownership boundaries, not by a target registration count. Independent accounts have separate logical locks, but share database CPU/I/O, connections, object storage and external service quotas. A single account with a large import can stress a different path than thousands of occasional readers.

Pressure point	Current design cost	Evidence-led next step
Full histories and streaks	All retrieved rows can accumulate in client memory; repeated scans grow with history	Date-bounded/virtualized UI, server aggregates and incremental invalidation, validated against full recomputation
SQLite account JSON	Every change can rewrite a large cache/outbox payload	Normalize records and operations while preserving one atomic local transaction
Per-account writes	Serialization favors correctness; hot-account imports can contend	Measure lock waits, then consider batching/narrower locks with concurrent invariant tests
Receipts/tombstones	No pruning protocol	Define recovery horizon, cursor reset/full resync and compatible clients before compaction
Photos	Bytes/egress dominate structured-record size	Quotas, orphan reconciliation, bounded delivery and Storage backup/restore
AI and barcodes	Provider latency, cost, cache misses and burst traffic	Request metrics, global/per-user budgets, burst controls, measured cache policy and idempotent job handling if needed

Concrete capacity test design

Generate synthetic users with realistic mixes: brief daily logging, multi-year history reads, same-account concurrent edits, backlog sync, images and bounded AI calls. Increase concurrency in staging while measuring p50/p95/p99 latency, failed/duplicate/lost writes, conflict rate, sync lag, lock wait, memory, database resource use, egress and spend.

Define service objectives before the run and choose launch capacity with headroom from actual results. Example acceptance properties are no cross-account reads, no accepted-save loss after retry, correct conflict reporting and a usable first screen/export at the tested data size. These are proposed gates, not measured current achievements.

Cost reasoning without invented prices

Estimate database/storage/egress, provider input/output tokens and request volume separately. A per-user daily quota does not cap total cost as user count grows. Add a global spend backstop and failure alerts. Database backups do not contain Storage image bytes; restore both and validate the cursor protocol. [R15]

Queues can support long exports or durable AI jobs. Redis/microservices require measured needs that justify their additional failure modes.

Evidence: HARDENING.md; pagination.ts; sqlite-store.ts; mutation/AI/photo code; scripts/check-hardening-concurrency.mjs. Scaling proposals are not implemented capacity claims.

Prioritized findings and improvement plan

The review updates resolved findings and identifies concrete remaining work; it does not silently change the app to fit the report.

Priority	Finding / action	Why it matters
Before claiming complete portability	Fix export coverage, lower-cap termination and documented consistency contract; retain versioned format	Prevents silently incomplete archives and misleading backup claims
Before public beta	Complete native HealthKit, Face ID, reminders, photos, keyboard/accessibility and device-cleanup checks	Browser and SQL evidence stop at native/service boundaries
Before public beta	Run independent concurrent database connections and live two-account Auth/RLS/Storage tests	Isolated serialized SQL cannot establish real race behavior or full hosted policy configuration
Before release	Align HealthKit purpose text; inspect iPad scope, email/redirect/session policy, privacy/support/AI disclosures	Configuration and distribution determine the actual product contract
Security/lifecycle follow-up	Review generic SecureStore fallback for deletion recovery capability; evaluate standard alternatives to custom biometric exchange	Avoid treating a convenience abstraction as a universal secure-storage guarantee
Before substantial growth	Profile history, SQLite rewrites, imports/exports and per-account locks	Correct complete reads can still exhaust memory or create poor latency
Operational hardening	Add privacy-filtered telemetry, rate/burst/global-cost controls, orphan cleanup and tested backups	Per-user quotas and individual retries do not replace operations
Maintenance	Document current visible UI separately from retained Coach/streak compatibility code; retire only with a migration/client plan	Reduces stale claims and unintended bypasses without discarding historical data

Already addressed since the earlier report

Account deletion exists and has documented hosted verification. Streak initialization has a deployed RPC fix. Logging/food-completion streaks and the PR widget are retired. Onboarding shares persistent helpers; entry dates and grouped settings are implemented. These should be demonstrated as completed work, with remaining native verification stated separately.

Architecture debt worth discussing honestly

The custom Face ID exchange and bespoke synchronization layer are significant complexity for a personal product. Their educational/reliability value is real, but a fresh implementation could choose standard authentication and a narrower offline scope. Several screen/repository modules are large; extracting cohesive state machines and shared query behavior may improve maintenance more than splitting backend services.

Recommended next milestone

An evidence-backed private beta: repair export claims/behavior, validate actual platform boundaries, stabilize release configuration and collect performance/error measurements. Use those results to prioritize further infrastructure work.

Evidence: Current code findings on pages 10-12, 15, 19, 26 and 28-32; repository deployment/test records; no application fix or hosted operation performed in this task.

Interview answer drafts: project and choices

These first-person drafts are grounded in the reviewed design. Use them as preparation, not as a claim of experiments or ownership you cannot substantiate.

'Tell me about this project.'

'I built an iPhone-first wellness tracker initially for my own lifting, food, fluids, weight and blood-pressure workflow. I wanted React/TypeScript experience, but I also needed native HealthKit and device features, so I used React Native with Expo. Supabase gave me relational storage, authentication and private objects without operating a separate general API server. The interesting engineering work was making saves trustworthy: offline vital writes, durable retries, transactional workouts and explicit conflict handling. I kept future accounts in mind through ownership boundaries, without claiming production-scale capacity.'

'Why not just use Swift or a web app?'

'SwiftUI would have been a strong iPhone-only choice. React Native matched my learning goals and let me share UI structure and domain logic with a browser preview while still using native adapters. A web-only app would not give me the same HealthKit path. I accepted dependency/native-debugging overhead and still need real-device tests; browser success is not native certification.'

'Did you overengineer a personal app?'

'Some investments were small and foundational: account IDs, RLS, constraints and feature boundaries. Others, especially custom biometric credentials and the outbox/conflict engine, were substantial. I would not justify them by imaginary millions of users. They solved real recovery/security requirements and gave me systems experience, but I would re-evaluate standard auth and a narrower offline scope if starting again. I deferred server reminders and distributed infrastructure to keep operations manageable.'

'Why PostgreSQL/Supabase?'

'My data has relationships and multi-row invariants: sessions and sets, owned photos, historical nutrition snapshots. PostgreSQL lets the final write enforce them in one transaction. Supabase reduced infrastructure setup and exposed authenticated/RLS-backed access. The tradeoff is platform coupling and more logic in SQL functions. A custom API becomes attractive if integration or team boundaries justify another service.'

'How did you plan for future users?'

'I designed ownership and retries before adding capacity infrastructure. Accounts have isolated records and local keys; database constraints enforce parent ownership; versions detect competing edits. That avoids a major data-model rewrite for another account. I have not benchmarked a maximum user count. My next steps are native acceptance, real concurrency tests and profiling the history/photo/AI workload, then optimizing the measured bottleneck.'

Useful follow-up: Show one real source/test example after the short answer. Specific failure behavior is more persuasive than listing framework names.

Evidence: User-stated motivations; architecture and evidence established in preceding chapters. These are adaptable answer drafts, not verbatim historical decision records.

Interview answer drafts: difficult follow-ups

Use a concrete invariant, a failure scenario, the mechanism, and its limitation. That structure exposes engineering judgment.

'How do you prevent duplicate saves?'

'I persist an operation ID and frozen request before sending. The database commits the data and receipt together. If the response is lost, retrying that same operation returns the receipt. I reject reuse with changed content. This is idempotent mutation behavior, not exactly-once delivery for external services such as image storage or an AI call.'

'Why not last-write-wins?'

'Two devices can edit the same workout. If both started at version 4, only one should replace it successfully. The next edit must compare against version 5 instead of erasing it. Goal updates compare individual changed fields so a calorie edit and a fluid edit can merge. The cost is a conflict UI and some user review.'

'How does offline sync avoid losing newer edits?'

'Cache and outbox commit together. Queued operations are immutable and linked to their base/dependency, so an old acknowledgment removes only its operation. Remote pages and their cursor also commit together. Tombstones preserve deletions. I have tests for failed storage and stale acknowledgments, but I still need real-device and independent-connection testing.'

'How can deletion recover after you revoke the session?'

'Preparation proves password and identity, then stores a hashed recovery capability and blocks further writes. Cleanup removes object bytes, deletes Auth and verifies remaining data. The saved capability can resume that prepared job without an ordinary session. The device clears its marker only after confirmed completion/cancellation. This handles partial work across systems that cannot share one transaction.'

'How do you keep AI from corrupting data?'

'The model proposes structured content, not arbitrary database writes. I restrict context/tools, validate output and recalculate deterministic values in code. Meal estimates require review; workout generation populates editable drafts. A normal tracker save is still required to record completion. Schema validity does not prove nutritional or training quality, so I need representative evaluation beyond mock tests.'

'What was a meaningful UI bug?'

'Selecting streak habits changed the live preview height above the choices, shifting the sheet. I separated preview into its own view and kept configuration geometry stable. Existing browser regressions assert unchanged bounds and scroll offset. For onboarding, I reduced competing keyboard-resizing containers to one inset-aware scroll owner, while keeping native validation open.'

'What would fail first at scale?'

'I would investigate full-history loading and SQLite account-payload rewrites before inventing a microservice bottleneck. Photos and AI could dominate cost. I would measure latency distributions, memory, lock waits, sync backlog and provider spend, then add bounded queries, normalized local rows or background jobs where the evidence supports them.'

Evidence: mutation-model.ts and tests; vitals/store-model.ts and SQLite tests; deletion implementation; AI boundary code; summary-refinements and onboarding browser regressions.

Decision checklist and interview guardrails

For each decision, know the benefit, accepted cost, strongest alternative and condition that would make you reconsider it.

Decision	Accepted cost / revisit when
React Native + Expo	Native bridges/build compatibility and device testing; reconsider if iOS-only platform depth dominates shared React value
Hosted Supabase	Vendor/API coupling and SQL business logic; add a dedicated service when versioned integration/team boundaries require it
RLS + guarded transactional RPCs	Policies/grants/functions need careful review; preserve database invariants even if an API tier is added
Process-only sessions	Online cold-start login; revisit convenience/offline expectations and standard auth options
Custom biometric credential exchange	Security and recovery complexity; evaluate a supported standard mechanism before broad distribution
Offline vitals, online-confirmed trackers	Uneven offline capability; expand only with explicit conflict and recovery semantics
Immutable nutrition/goal snapshots	Duplicated data and explicit correction work; necessary when historical meaning must remain stable
Derived streaks	Complete-history cost; cache/materialize only with source-edit/deletion invalidation
Device-local layouts/reminders	No cross-device preference sync; add cloud delivery only when its operational cost is justified
Reviewed AI drafts	Extra review and generation latency; retain user control even when model quality improves
Bounded processed photos	Less image fidelity; tune only after privacy/storage/quality requirements are measured
No early microservices/Redis/queues	Some work remains synchronous/client-heavy; introduce components for demonstrated bottlenecks

Claims to avoid

- 'Everything works offline' or 'Face ID encrypts all app data.'
- 'Exactly once,' 'fully atomic photos,' or 'instant revocation everywhere.'
- 'AI is accurate because JSON validation passes' or 'meeting a streak proves complete intake.'
- 'Production-ready for 10,000 users' without a repeatable workload and measured capacity.

Questions you should be ready to answer at a whiteboard

Where is the authoritative value? Which operations share one transaction? What survives a crash? What happens after a timeout? Which account owns a queued operation? What makes a cursor safe? Which historical values change after an edit? What is the first measurement you would collect before optimizing?

The strongest answer often includes a limitation you discovered and a precise next experiment. That demonstrates judgment more convincingly than defending every implementation choice as permanent.

Evidence: Synthesis of reviewed implementation, user-stated goals, ADRs and concrete findings. No comparative benchmark or hiring-market statistics are asserted.

Code evidence map and reproducibility

Paths below are relative to the repository. File hashes in the companion manifest identify the local snapshot, including code absent from the baseline commit.

Area	Main source anchors
App and distribution	package.json; apps/mobile/package.json; app.json; app.config.ts; eas.json; app/(app)/_layout.tsx; .github/workflows/ci.yml
Auth and deletion	apps/mobile/src/features/auth/{auth-provider,biometric-auth,account-deletion,security-settings}; src/lib/{supabase,secure-store}; supabase/functions/{biometric-auth,delete-account}; migrations 202609180003 and 202609220002
Save/sync	apps/mobile/src/lib/{mutation-model,mutations,pagination}.ts; features/vitals/{store-model,sqlite-store,sync,storage.native,storage.web}; migration 202609180004
Vitals and HealthKit	src/domain/vitals.ts; features/vitals/{chart,blood-pressure>manual-vitals-log}; features/healthkit/{reader.ios,sync,identity,permissions,unified-sync}
Training	features/training/{repository,progression,workout-draft,cardio-draft,workout-history}; features/summary/training.ts; app/(app)/workout.tsx
Food and fluids	features/nutrition/{model,repository,recipe,draft,history-totals}; features/hydration/{model,categories,repository}; functions/resolve-food-barcode; functions/_shared/hydration.ts
AI	app/(app)/coach.tsx; features/coach/{workout-questionnaire,draft-actions}; features/nutrition/ai-meal*; functions/{estimate-meal,coach-chat}; _shared/{meal-estimate,workout-planning,coach-action-validation}
Goals, Profile, setup	features/goals/{calculator,helper-model,helper-draft,use-helper-state,repository}; features/profile/*; features/onboarding/*; src/lib/entry-date.ts
Dashboard/streaks	features/summary/{layout,layout-store,data,dashboard,editor-grid,streak-week}; features/streaks/{engine,repository,initialize}; ADR-008 amendments
Reminders	features/reminders/{routine-model,schedule-model,service,device.native,navigation-model,notification-observer,intent-storage}; ADR-009
Photos/export	features/progress-photos/{image,model,repository}; features/data-export/{repository,model,archive.native,archive.web}; ADR-003 and hardening amendments
Verification	src/**/*.test.ts; supabase/functions/**/*.test.ts; supabase/tests; scripts/check-onboarding-sql.mjs; selected e2e suites; report-review logs

Review method

Read the original 16-page report; reconciled routes, schemas, repositories, migrations, pure models, native adapters, ADR amendments and deployment records; reran local checks; consulted primary framework/platform/scientific references; produced and visually verified this replacement report. No health export, secret file or real personal record was used in the report.

Markdown is editable; HTML preserves diagrams. The manifest is an index, not a source archive. Preserve or commit the reviewed working tree for reproduction.

Evidence: Baseline HEAD e39d7c003a9a3324e55d62e15398265b5d088fe0 plus uncommitted files; HealthApp_Report_Evidence_Manifest.json.

Primary references and terminology

External sources validate platform behavior and equation provenance. They do not independently verify HealthApp code, deployments, medical suitability or capacity.

Framework, database and platform references

- [R1] [React Native core/native components](#) - component model and native view mapping.
- [R2] [Expo development builds](#) - custom native runtime versus Expo Go.
- [R3] [Supabase Row Level Security](#) - database authorization boundary.
- [R4] [PostgreSQL explicit locking](#) - transaction/row/advisory lock semantics.
- [R5] [Supabase sessions](#) - sessions and hosted policy considerations.
- [R6] [Apple HealthKit authorization](#) - privacy limitations of inferring read permission.
- [R10] [Expo SDK 57 Notifications](#) - native scheduling and response APIs.
- [R11] [Supabase Storage access control](#) - object authorization distinct from table metadata.
- [R12] [Expo iOS submission](#) - build upload and distribution workflow.
- [R13] [Apple account deletion guidance](#) - in-app deletion initiation for account-creating apps.
- [R14] [Apple App Review Guidelines](#) - privacy, health and review requirements.
- [R15] [Supabase database backups](#) - database recovery does not include stored object bytes.

Equation and fluid reference context

- [R7] [Mifflin et al., 1990](#) - simplified resting-energy equation; not validation of all app input bounds, activity factors or weight-loss forecasts.
- [R8] [NIDDK Body Weight Planner](#) - planning limitations/lower-intake warning; its dynamic model is not the app's fixed-deficit arithmetic.
- [R9] [National Academies, Water chapter](#) - population water/beverage references; the app's 250/500/750-mL activity increments are separate product assumptions.

Terms used in this report

RLS: Row Level Security. **RPC:** a remotely invoked database function. **CAS:** compare-and-set against the version/value originally edited. **Outbox:** durable local operations waiting for sync. **Tombstone:** retained deletion marker. **Idempotent retry:** repeating the same operation does not repeat its committed effect. **Keyset pagination:** continue after a stable key rather than a shifting offset. **Capability token:** possession authorizes a narrowly scoped continuation. **PGlite:** isolated PostgreSQL runtime used here with platform adapters, not a hosted concurrency environment.

References accessed September 23, 2026. Comparative design analysis is the review's reasoning; historical decisions are attributed to ADRs and user-stated intent. App policy thresholds are distinguished from validated clinical recommendations.